



Advancement Report

AI-Powered Intrusion Detection System for Cloud Environments

KRID Aboubakr

Master of Artificial Intelligence and Cybersecurity

Supervised by Prof. Fakhri Yousef

May 2026

Contents

1	Introduction and Context	2
2	Host Machine Assessment	3
2.1	Hardware Specifications	3
2.2	Why These Specs Matter for the Project	3
2.2.1	Linux as the Host OS	3
2.2.2	Hardware Virtualisation Confirmed	3
2.2.3	RAM: Sufficient but Requires Careful Allocation	3
3	Storage Architecture	4
3.1	The Problem with the Default Layout	4
3.2	The Dual-Partition Solution	4
3.3	Final Storage Layout	4
4	Virtualisation Strategy	5
4.1	Why Not OpenStack?	5
4.2	Why Two VMs Rather Than Mininet?	5
4.3	Why KVM Rather Than VirtualBox?	6
5	Virtual Machine Design	7
5.1	VM1 Ubuntu Server 22.04 (Target + IDS Brain)	7
5.1.1	Why 5 GB RAM	7
5.1.2	Why 4 vCPUs	7
5.1.3	Why Ubuntu Server (no desktop)	7
5.1.4	Why 35 GB Disk	7
5.2	VM2 Kali Linux 2026.1 (Attacker Machine)	8
5.2.1	Why Xfce Desktop	8
5.3	Network Architecture	8
5.3.1	Routing Challenge and Resolution	8

1. Introduction and Context

This document is the first of two companion reports produced at the end of the environment setup phase of the EYP project. While the second report (the Technical Report) documents every command and configuration file used, this report focuses on *why* each decision was made the reasoning, trade-offs, and academic justification behind the chosen architecture.

The project goal, as defined in the Advancement Report submitted in April 2026, is to design and implement an AI-powered Intrusion Detection System (IDS) adapted to cloud environments. The system must cover the full pipeline: from raw network traffic capture to machine learning-based detection, real-time alerting, and an explainability layer that makes each alert actionable for a security analyst.

Before any code could be written or models trained, a reproducible, isolated lab environment had to be constructed. This phase is documented here.

2. Host Machine Assessment

2.1 Hardware Specifications

The development host is a laptop with the following specifications:

Component	Specification
CPU	Intel Core i7-13620H
RAM	16 GB DDR5
GPU	NVIDIA GeForce RTX 4050 Mobile (CUDA support)
Storage	476 GB NVMe SSD
OS	Pop!_OS 24.04 LTS (Ubuntu-based)

2.2 Why These Specs Matter for the Project

2.2.1 Linux as the Host OS

Running Pop!_OS (an Ubuntu derivative) natively is a significant advantage. It means Docker runs with zero virtualisation overhead, and KVM/QEMU, the Linux-native hypervisor can be used instead of VirtualBox. KVM is built into the Linux kernel and offers near-native performance for virtual machines because the hypervisor is part of the kernel itself rather than a user-space application.

2.2.2 Hardware Virtualisation Confirmed

The presence of the `vmx` flag in `/proc/cpuinfo` (32 entries confirmed) means the CPU supports Intel VT-x. This is a prerequisite for KVM and allows virtual machines to execute privileged instructions directly on the hardware rather than emulating them in software. Without this flag, virtual machine performance would be unacceptably slow.

2.2.3 RAM: Sufficient but Requires Careful Allocation

16 GB is workable for this setup. The allocation across the system is:

- Host OS: approximately 5.5 GB at idle
- VM1 (Ubuntu Server): 5 GB assigned
- VM2 (Kali Linux): 3 GB assigned
- Buffer: approximately 2.5 GB

3. Storage Architecture

3.1 The Problem with the Default Layout

The original disk layout presented a storage challenge. The NVMe drive contained a Windows dual-boot installation occupying the majority of space, leaving only approximately 55 GB free on the Pop!_OS root partition (`/dev/nvme0n1p8`). Two virtual machine disk images (estimated 25–30 GB of actual usage) plus Docker image layers would have filled the root partition during the project.

3.2 The Dual-Partition Solution

The Windows D: partition (`nvme0n1p4`) had already been shrunk from 144.9 GB to 64.7 GB from Windows Disk Management, freeing approximately 80 GB of unallocated space. This unallocated space was formatted as a dedicated ext4 partition (`/dev/nvme0n1p9`, labelled `lab-data`) and mounted permanently at `/mnt/lab`.

Why NTFS cannot be used for infrastructure

The project folder resides on the D: partition (NTFS). While NTFS is suitable for storing Python source code, CSV datasets, and trained model files, it cannot be used for KVM disk images (qcow2 requires sparse file support and Linux file permissions) or Docker volumes (containers rely on `chmod/chown` which NTFS silently ignores). The ext4 partition is therefore essential for all infrastructure components.

3.3 Final Storage Layout

Partition	Size	Filesystem	Purpose
<code>nvme0n1p3</code>	209.9 GB	NTFS	Windows C:
<code>nvme0n1p4</code>	64.7 GB	NTFS	D: project code, CSVs, model files
<code>nvme0n1p8</code>	99.0 GB	ext4	Pop!_OS root (<code>/</code>)
<code>nvme0n1p9</code>	79.0 GB	ext4	<code>/mnt/lab</code> Docker, VMs, datasets

The `/mnt/lab` partition is subdivided into three directories:

- `/mnt/lab/docker` Docker data root (images, containers, volumes)
- `/mnt/lab/vms` KVM virtual machine disk images (qcow2)
- `/mnt/lab/datasets` CICIDS-2017 dataset files

4. Virtualisation Strategy

4.1 Why Not OpenStack?

The project’s academic motivation, as stated in the Advancement Report, is to build an IDS for organisations running private clouds such as OpenStack. A natural question is whether the development should occur directly on an OpenStack installation.

The answer is no, for three practical reasons:

1. **Hardware requirements.** A minimal stable OpenStack deployment requires a dedicated server with at least 32 GB of RAM. Running it on a 16 GB laptop alongside development tools is not feasible.
2. **Setup complexity.** OpenStack consists of approximately 15 interdependent services (Nova, Neutron, Cinder, Keystone, Glance, etc.). Installation using DevStack or Packstack typically takes 2–5 days and is subject to frequent failures on updates. This represents an unacceptable risk for a time-constrained master’s project.
3. **Misallocation of effort.** The academic contribution of this project is the explainability layer on top of a working ML-IDS pipeline. Spending weeks on OpenStack configuration would consume time that should be spent on the IDS itself.

The defensible academic position is that the system is designed and validated on a simulated cloud environment that replicates the network flow characteristics of a private cloud. OpenStack deployment remains a natural extension for future work.

4.2 Why Two VMs Rather Than Mininet?

An earlier consideration was to use Mininet (a Python-based network emulator) inside a single VM to simulate the attacker/victim topology. This approach was rejected in favour of two separate VMs for the following reasons:

- **Traffic realism.** When Kali Linux’s Hydra tool brute-forces SSH on VM1, the packets crossing the internal network are genuine TCP handshakes, not simulated flow statistics. CICFlowMeter on VM1 captures and converts these into the same feature format as CICIDS-2017, making the live demo directly comparable to the training data.
- **Demo quality.** The two-VM setup allows a compelling visual demonstration: VM2 running an attack script while VM1’s Streamlit dashboard fires alerts in real time. This is far more convincing than synthetic packet replay.

- **Architectural honesty.** Real IDS systems sit on the same machine as the services they protect. VM1 running both real services and the IDS watching them mirrors this accurately.

4.3 Why KVM Rather Than VirtualBox?

KVM (Kernel-based Virtual Machine) is the Linux-native hypervisor, built into the kernel since version 2.6.20. On a Linux host, KVM outperforms VirtualBox because:

- There is no user-space translation layer virtualised instructions execute directly via the kernel
- virtio network drivers provide near-native network throughput, which matters for traffic capture
- Thin-provisioned qcow2 disk images only consume actual written space on the NVMe, not the full allocated size

5. Virtual Machine Design

5.1 VM1 Ubuntu Server 22.04 (Target + IDS Brain)

VM1 serves a dual role: it runs the real services that Kali will attack (Apache, SSH, FTP), and it hosts the entire IDS pipeline inside a Docker Compose stack.

VM1 Resource Allocation

RAM:	5 GB
vCPUs:	4
Disk:	35 GB (thin-provisioned qcow2)
eth0:	ids-lab network (192.168.100.46)
eth1:	default NAT (192.168.122.206)

5.1.1 Why 5 GB RAM

The peak memory consumption during a live demo includes Ubuntu Server OS (≈ 400 MB), Docker daemon (≈ 200 MB), Kafka + Zookeeper (≈ 800 MB), detection engine (≈ 400 MB), MLflow (≈ 300 MB), Streamlit (≈ 300 MB), PostgreSQL (≈ 200 MB), CICFlowMeter Java heap (≈ 512 MB), and system buffers (≈ 500 MB). Total peak usage is approximately 3.6 GB. The 5 GB allocation provides 1.4 GB of headroom, preventing Kafka out-of-memory crashes during sustained attack simulation.

5.1.2 Why 4 vCPUs

CICFlowMeter is a Java application that processes pcap files and is CPU-intensive. With 2 vCPUs, converting a 500 MB capture file takes several minutes and becomes a bottleneck. With 4 vCPUs, the conversion is noticeably faster. Kafka also benefits from multiple cores for its I/O threads.

5.1.3 Why Ubuntu Server (no desktop)

Ubuntu Server is terminal-only by design. This is correct for three reasons: a GNOME desktop consumes 400–600 MB of RAM that is better spent on containers; real cloud servers are always headless; and VM1 is managed via SSH from the host, which is cleaner than using a graphical console.

5.1.4 Why 35 GB Disk

The initial recommendation was 25 GB. This was revised to 35 GB because pcap files generated during attack simulation can grow rapidly (500 MB per 10-minute session). The

thin-provisioned qcow2 format means the extra space costs nothing on the NVMe until it is actually written.

5.2 VM2 Kali Linux 2026.1 (Attacker Machine)

VM2 is the attack machine. Kali Linux was chosen because every attack tool needed for the project (Nmap, Hydra, Metasploit, hping3, Slowloris, Wireshark) comes pre-installed and maintained by the Kali team. This eliminates a week of manual tool installation and configuration.

VM2 Resource Allocation

RAM:	3 GB
vCPUs:	2
Disk:	30 GB (thin-provisioned qcow2)
eth0:	ids-lab network (192.168.100.28)
eth1:	default NAT (192.168.122.219)

5.2.1 Why Xfce Desktop

Kali's Xfce environment uses approximately 200 MB RAM at idle, compared to 500 MB for GNOME and 400 MB for KDE. On a VM with only 3 GB assigned, this difference matters. Xfce is also Kali's official default desktop for exactly this reason.

5.3 Network Architecture

Two virtual networks were created using libvirt:

Network	Subnet	Purpose
ids-lab	192.168.100.0/24	Isolated attack/capture traffic
default	192.168.122.0/24	NAT for internet access

The `ids-lab` network is the critical one. All attack traffic from VM2 to VM1 crosses this network, and `tcpdump` on VM1 listens on the `eth0` interface connected to it. The `default` network provides internet access for package installation and updates and carries no project traffic.

5.3.1 Routing Challenge and Resolution

A routing conflict was encountered during Kali's network configuration. When both `eth0` (`ids-lab`) and `eth1` (`default`) were activated, each attempted to install a default gateway, causing the two interfaces to conflict and alternately lose connectivity.

The resolution was to use NetworkManager's `ipv4.never-default` property. Setting this to `yes` on `eth0` instructs NetworkManager that `eth0` should never be used as the default gateway it is for internal lab traffic only. `eth1` retains the default gateway role for internet traffic. A permanent connection profile was created for `eth1` using `nmcli connection add`, ensuring both interfaces activate correctly on every boot.